# REPORT ON FPGA BASED PASSWORD AUTHENTICATION SYSTEM

## Assignment -2

**Done By:**
Shreya Ashok Kulkarni
(3122243002106)
Ashvika Shalini Gowda
(3122243002302)
Navya Sree Nemali
(3122243002308)



# REPORT ON FPGA BASED PASSWORD AUTHENTICATION SYSTEM

## Aim:

To design and implement a 4-bit digital password lock system using Verilog HDL, where the entered password is compared with the stored password, the system unlocks for a correct match, and locks permanently after three unsuccessful attempts.

## Components Required:

FALSTAD
NEXYS FPGA board
Xilinx Vivado
Push buttons/switches for password inputs and attempt
LEDs for Unlocked and Locked indication
USB cable for FPGA programming

## Theory:

A digital password lock is a security circuit used to control access by comparing an entered password with a predefined password. In this experiment, the password lock is implemented using Verilog HDL on an FPGA.

The system uses two 4-bit inputs, a and b. One represents the entered password and the other represents the stored/reference password. The comparison is performed using:

$$\boxed{M = (a_3 \equiv b_3)(a_2 \equiv b_2)(a_1 \equiv b_1)(a_0 \equiv b_0)}$$

If both 4-bit values are equal, the password is considered correct and the Unlocked output is activated.

To improve security, the circuit also includes an attempt counter. Two T flip-flops are used to form a 2-bit counter that counts the number of password attempts. For every attempt, the counter changes its state. When the counter reaches 3, the circuit detects the condition using:

$$\boxed{\text{count_equals_3} = tff0_q \cdot tff1_q}$$

If the password is incorrect when the third attempt is reached, the Lock condition is generated and the locked output becomes HIGH.

The lock logic ensures that once the system is locked, it remains locked until a reset is applied. Similarly, when the system is locked, the unlocked output is forced LOW so that both security states do not occur simultaneously.

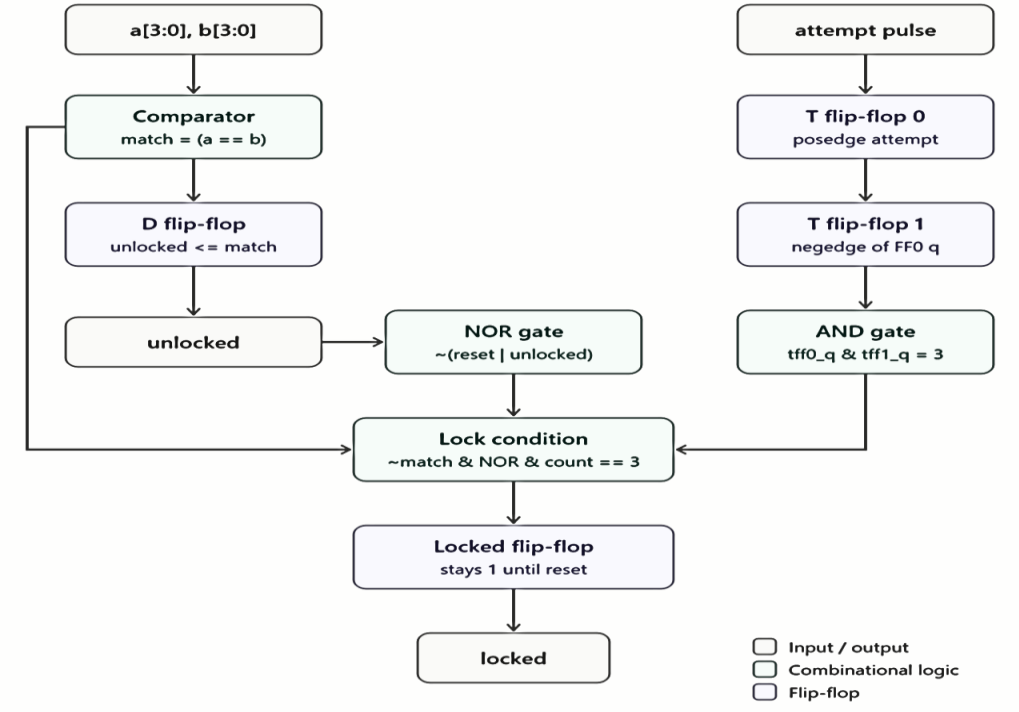
The circuit also uses NOR-based logic to control the lock condition. The clock signal is used to synchronize the unlocked and locked outputs, while the reset signal returns the counter and outputs to their initial states.

Thus, the complete system provides three main operations:

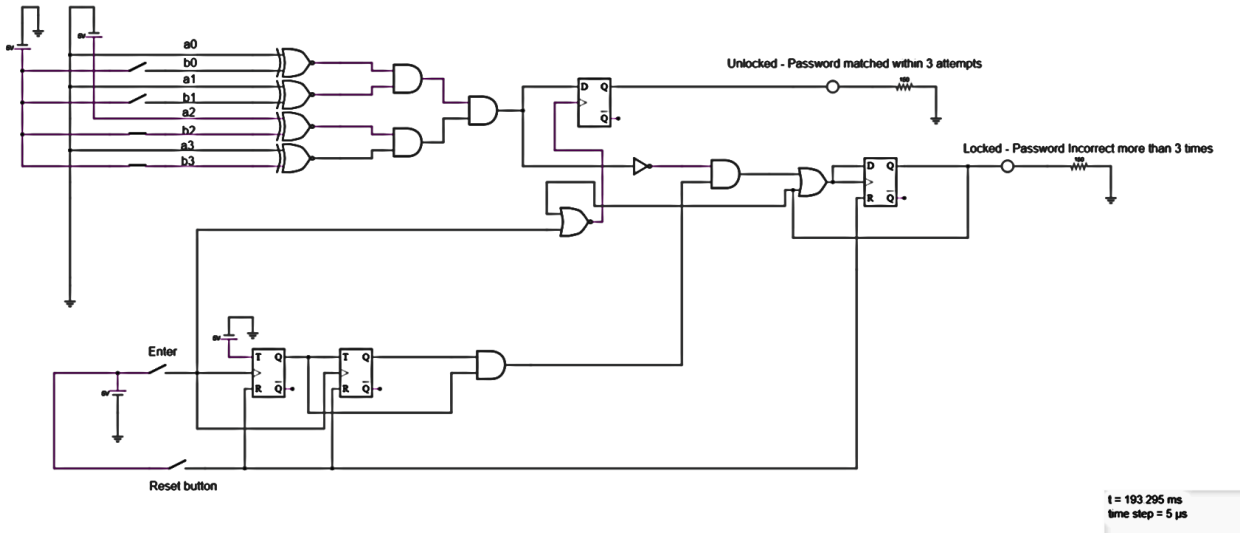
- Correct password → Unlocked
- Incorrect password → Attempt counter increases
- Third incorrect attempt → Locked
- Reset → System returns to initial state

The design demonstrates the use of password comparison, T flip-flops, counters, logic gates, clocked logic, and FPGA implementation in a simple digital security system.

## Flow Diagram:

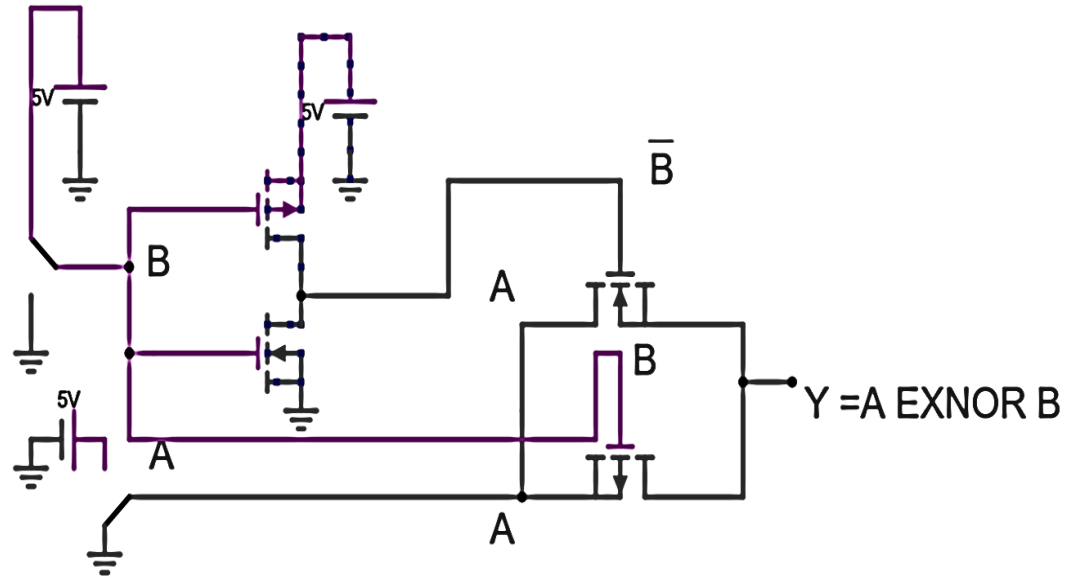


## Circuit diagram:

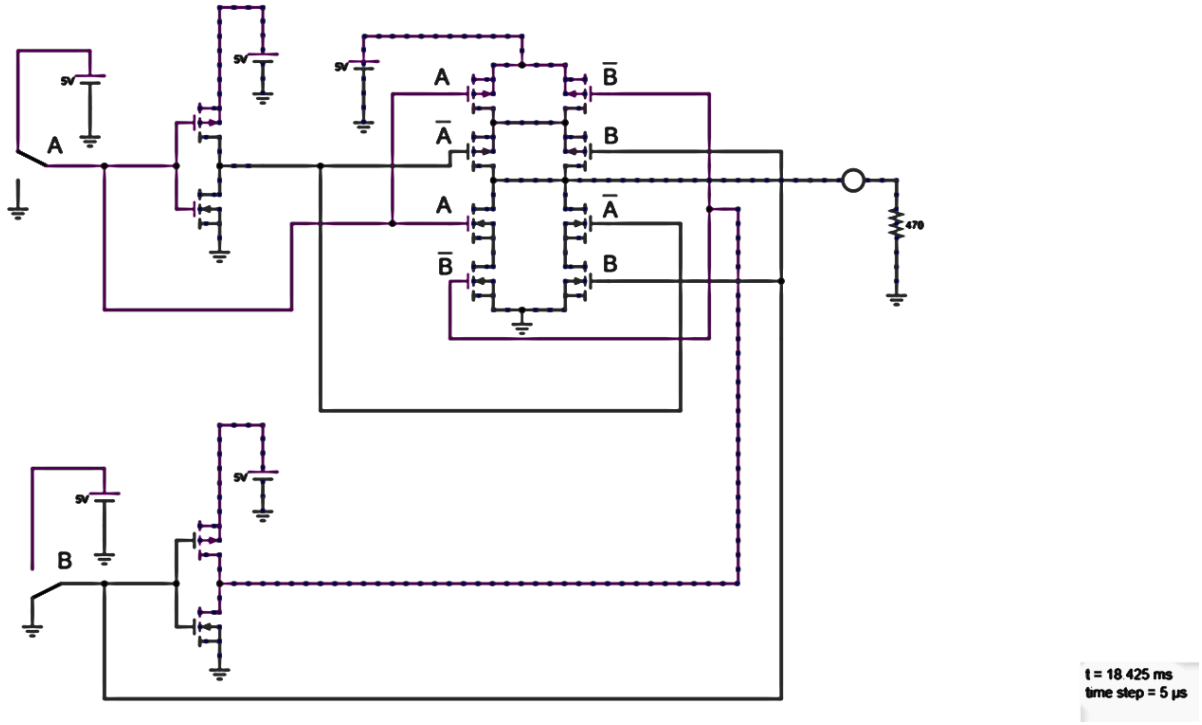



## Individual Blocks implemented using CMOS:

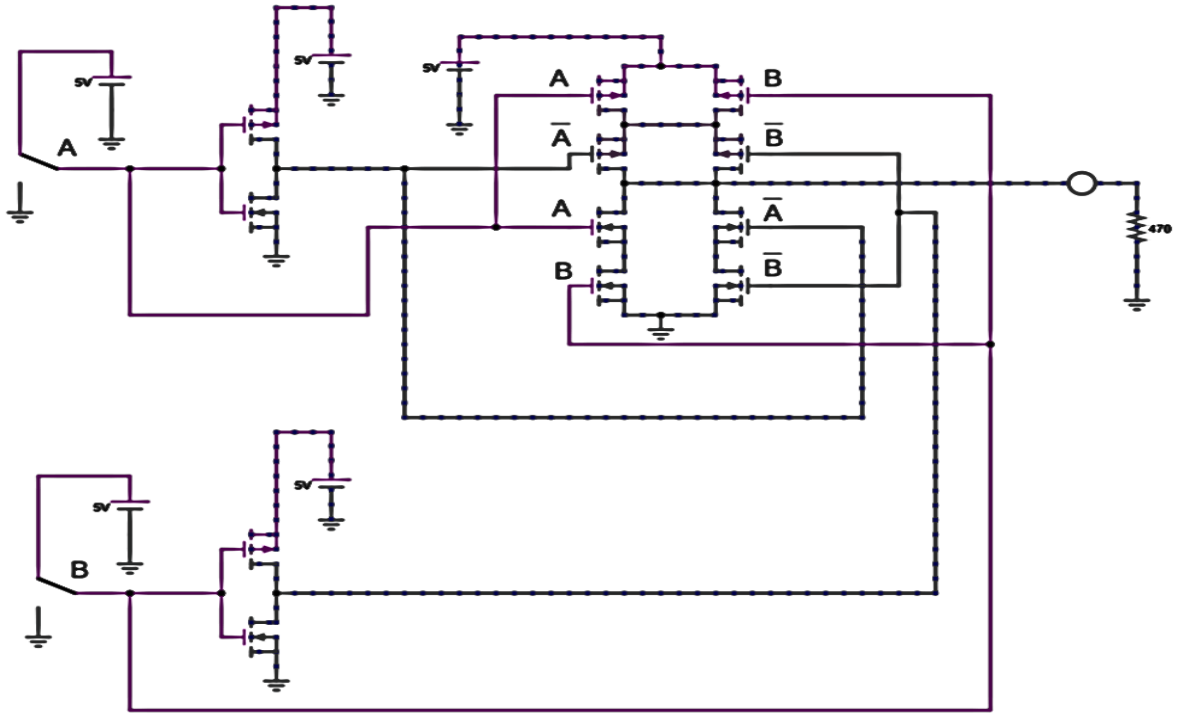
1. EXNOR using transmission gate



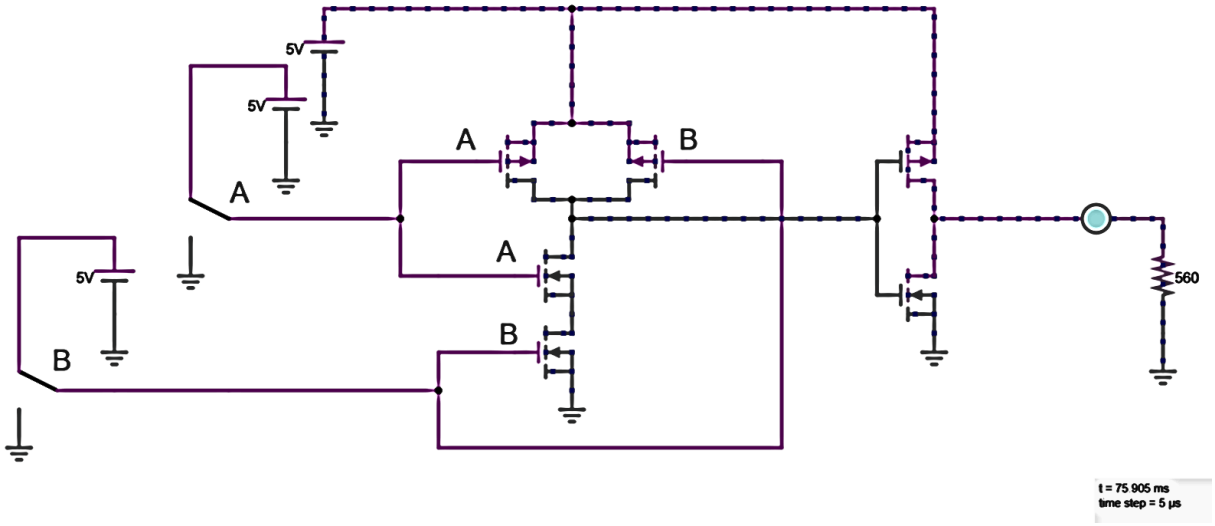
2. EXNOR using Static CMOS:



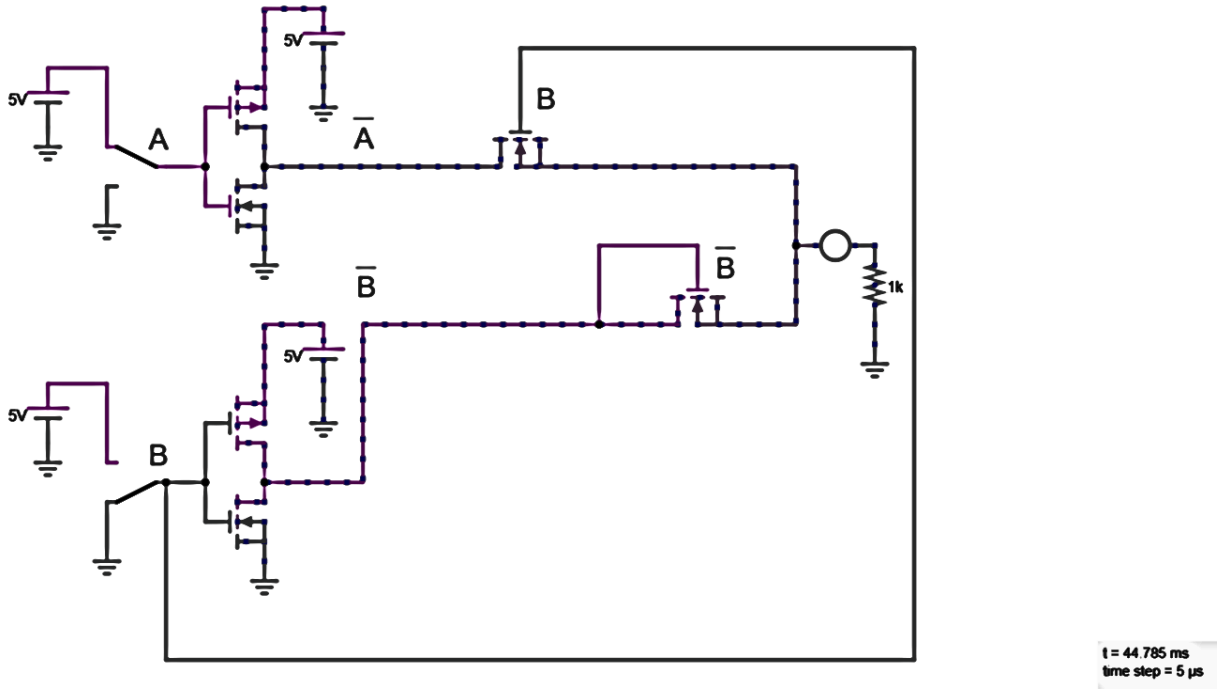

3. EXOR using Static CMOS:



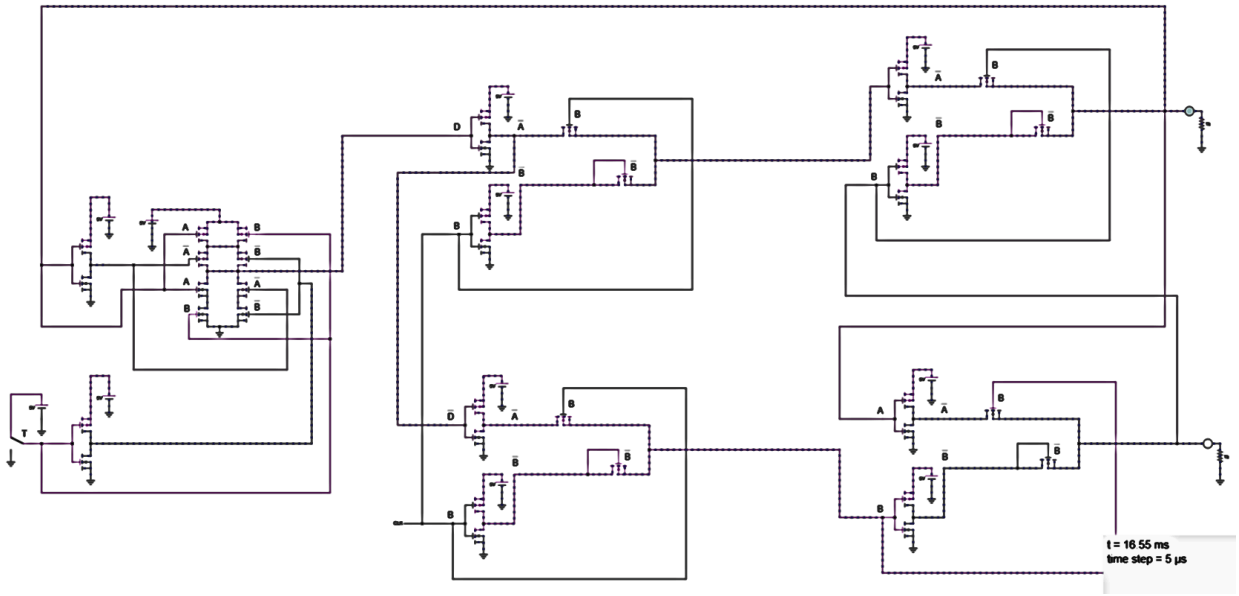
4. AND using Static CMOS:



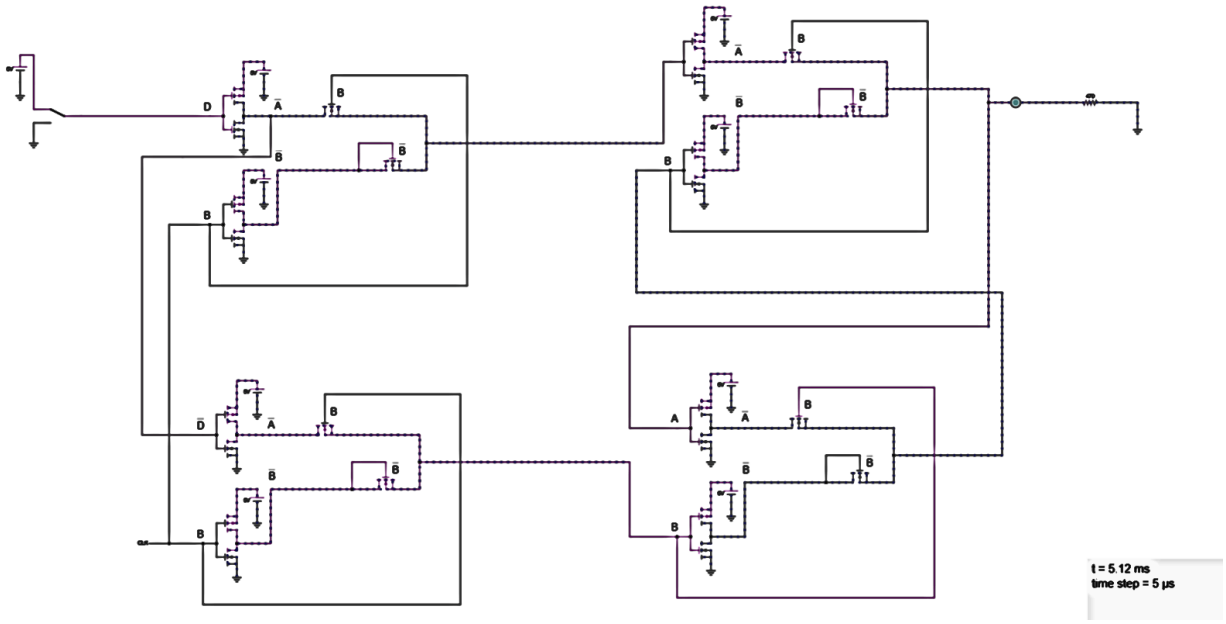

5. NAND using transmission gate:



6. T Flip-flop:



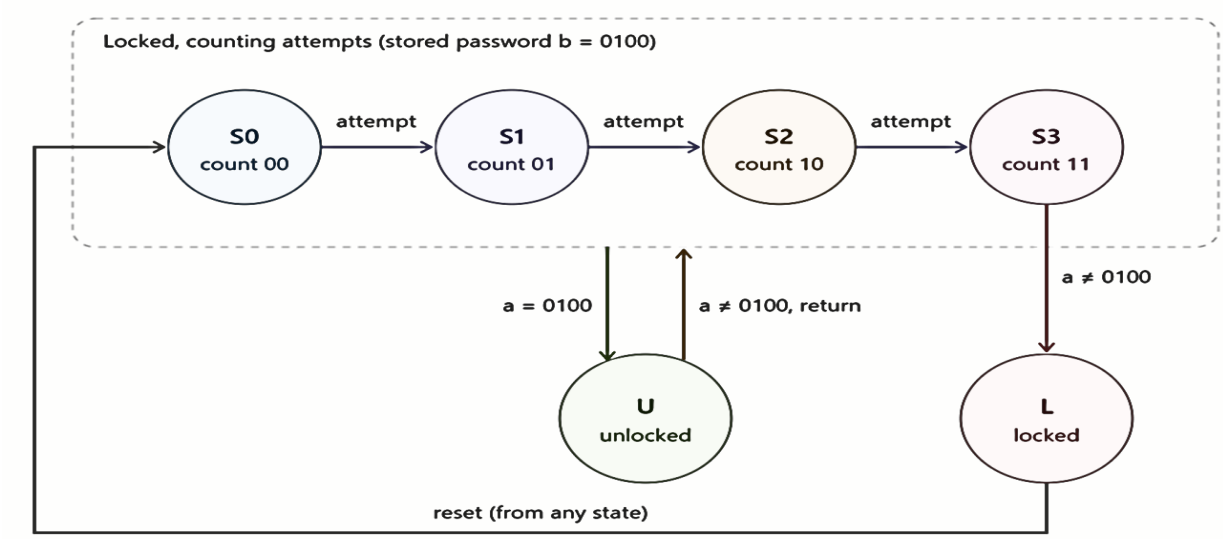

7. D Flip flop:



## Truth Table:

| Scenario | Reset | Enter | a | b | Password Status | Attempt No. | Unlocked | Freeze |
|---|---|---|---|---|---|---|---|---|
| 1. System Reset | **1** | 0 | 0 | 100 | — | 0 | **0** | **0** |
| 2. First Wrong | 0 | **1** | 1 | 100 | Incorrect | 1 | **0** | **0** |
| 3. Second Wrong | 0 | **1** | 110 | 100 | Incorrect | 2 | **0** | **0** |
| 4. Third Wrong | 0 | **1** | 1111 | 100 | Incorrect | 3 | **0** | **1** |
| 5. Correct While Frozen | 0 | **1** | 100 | 100 | Correct | 3+ | **0** | **1** |
| 6. Reset | **1** | 0 | 100 | 100 | — | 0 | **0** | **0** |
| 7. Correct After Reset | 0 | **1** | 100 | 100 | Correct | 1 | **1** | **0** |

## State Diagram:




## Verilog HDL Program:

```
module password_lock (
    input  wire       enter,      // Enter push button
    input  wire       reset,      // Reset push button
    input  wire [3:0] a,          // Entered password
    input  wire [3:0] b,          // Correct password
    output reg        unlocked,   // Green LED
    output reg        freeze      // Red LED
);

    // Password comparison
      wire match;
    assign match = (a == b);
```

```
    // Number of wrong attempts
    reg [1:0] attempt_count;
    // ================================================
    // ASYNCHRONOUS PASSWORD LOCK
    // ENTER acts as the event/clock
    // ================================================
    always @(posedge enter or posedge reset) begin
        if (reset) begin
            attempt_count <= 2'b00;
            unlocked      <= 1'b0;
            freeze        <= 1'b0;
        end
        // ------------------------------------------------
        // Already frozen
        // ------------------------------------------------
        else if (freeze) begin
            unlocked <= 1'b0;
            freeze   <= 1'b1;
        end
        // ------------------------------------------------
        // Correct password
        // ------------------------------------------------
        else if (match) begin
            unlocked      <= 1'b1;
            freeze        <= 1'b0;
            attempt_count <= 2'b00;
        end
        // ------------------------------------------------
        // Wrong password
        // ------------------------------------------------
        else begin
            unlocked <= 1'b0;
            // Third wrong attempt
            if (attempt_count == 2'b10) begin
                freeze <= 1'b1;
            end
            // First or second wrong attempt
            else begin
                attempt_count <= attempt_count + 1'b1;
                freeze        <= 1'b0;
            end
        end
    end
endmodule
```

## Stimulus File:

```
`timescale 1ns / 1ps
module tb_password_lock;
    // ================================================
    // Testbench Signals
    // ================================================
    reg       enter;
    reg       reset;
    reg  [3:0] a;
    reg  [3:0] b;
    wire      unlocked;
    wire      freeze;
    // ================================================
    // Instantiate Unit Under Test (UUT)
    // ================================================
    password_lock uut (
        .enter(enter),
        .reset(reset),
        .a(a),
        .b(b),
        .unlocked(unlocked),
        .freeze(freeze)
    );
    // ================================================
    // Task: Press ENTER button
    // One rising edge of ENTER = one password attempt
    // ================================================
    task press_enter;
        begin
            #10;
            enter = 1'b1;
            #10;
            enter = 1'b0;
            #10;
        end
    endtask
    // ================================================
    // Test Sequence
    // ================================================
    initial begin
        // ------------------------------------------
        // Initialize Inputs
```

```
        // ------------------------------------------------
        enter = 1'b0;
        reset = 1'b0;
        a = 4'b0000;
        b = 4'b0100;       // Correct password = 0100
        // ------------------------------------------------
        // Monitor Outputs
        // ------------------------------------------------
        $monitor(
            "Time=%0t | Reset=%b | Enter=%b | Target(b)=%b | Input(a)=%b | Unlocked=%b |
Freeze=%b",
            $time,
            reset,
            enter,
            b,
            a,
            unlocked,
            freeze
        );
        // ================================================
        // SCENARIO 1: SYSTEM RESET
        // ================================================
        $display("
--- Applying System Reset ---");
        reset = 1'b1;
        #20;
        reset = 1'b0;
        #10;
        // ================================================
        // SCENARIO 2: FIRST WRONG PASSWORD
        // ================================================
        $display("
--- Attempt 1: Incorrect Password ---");
        a = 4'b0001;
        press_enter();
        // ================================================
        // SCENARIO 3: SECOND WRONG PASSWORD
        // ================================================
        $display("
--- Attempt 2: Incorrect Password ---");
        a = 4'b0110;
        press_enter();
```

```
        // ================================================
        // SCENARIO 4: THIRD WRONG PASSWORD
        // SHOULD FREEZE THE SYSTEM
        // ================================================
        $display("
--- Attempt 3: Incorrect Password (Should Freeze) ---");
        a = 4'b1111;
        press_enter();
        #20;
        // ================================================
        // SCENARIO 5: CORRECT PASSWORD WHILE FROZEN
        // SHOULD NOT UNLOCK
        // ================================================
        $display("
--- Entering Correct Password While System is Frozen ---");
        a = 4'b0100;
        press_enter();
        #20;
        // ================================================
        // SCENARIO 6: RESET THE SYSTEM
        // ================================================
        $display("
--- Resetting System to Clear Freeze State ---");
        reset = 1'b1;
        #20;
        reset = 1'b0;
        #10;
        // ================================================
        // SCENARIO 7: CORRECT PASSWORD AFTER RESET
        // SHOULD UNLOCK
        // ================================================
        $display("
--- Attempting Correct Password After Reset ---");
        a = 4'b0100;
        press_enter();
        #30;
        // ================================================
        // END SIMULATION
        // ================================================
        $display("
Simulation Complete.");
        $finish;
```

```
    end
endmodule
```

## Procedure:

1. Open Xilinx Vivado and create a new RTL project for the required FPGA board.
2. Create a Verilog HDL source file and enter the password_lock module.
3. Use two 4-bit inputs, a and b , representing the entered password and stored/reference password.
4. The circuit compares both inputs using a == b.
5. If the passwords match, the unlocked output becomes HIGH.
6. Each incorrect attempt is counted using the 2-bit attempt counter made using T flip-flops.
7. When the counter reaches 3, the lock condition is activated for an incorrect password.
8. Once the locked output becomes HIGH, it remains HIGH until the system is reset.
9. Assign the input and output signals to the required switches, buttons, and LEDs using the XDC constraints file.
10. Run Synthesis → Implementation → Generate Bitstream in Vivado.
11. Program the generated bitstream onto the Nexys FPGA board.
12. Test the system by entering correct and incorrect passwords and observing the Unlocked and Locked LEDs.

## Video Demonstration:

Scan the QR code to see video demonstration of the project implementation in FPGA:

**Snapshots of the demonstration:**

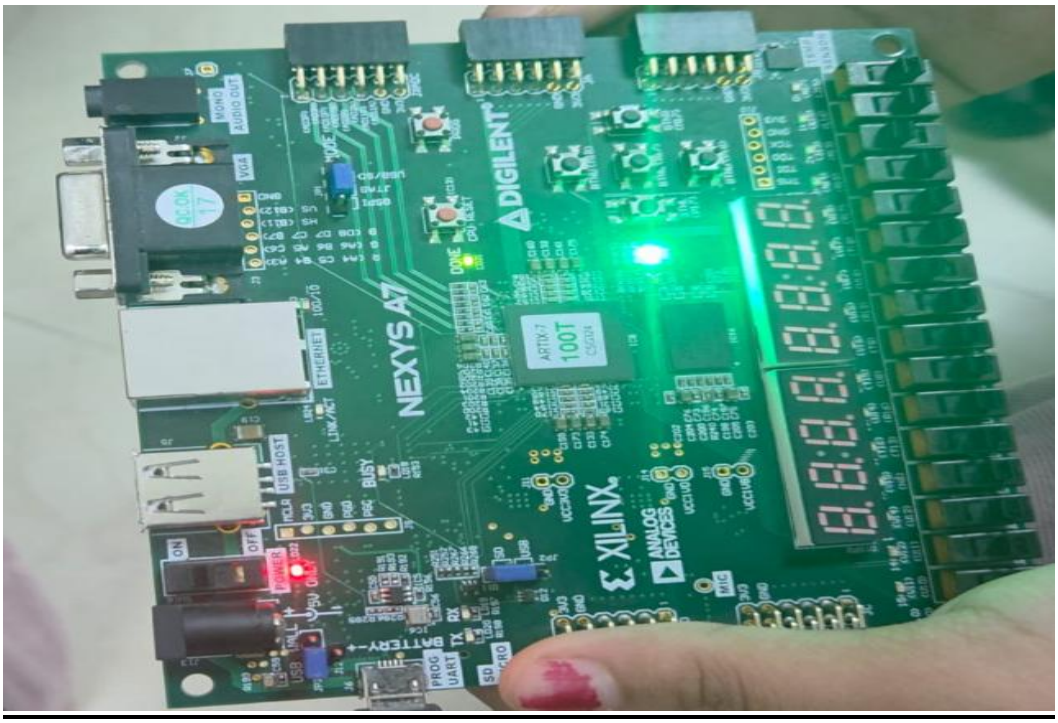



Fig. Output for correct password

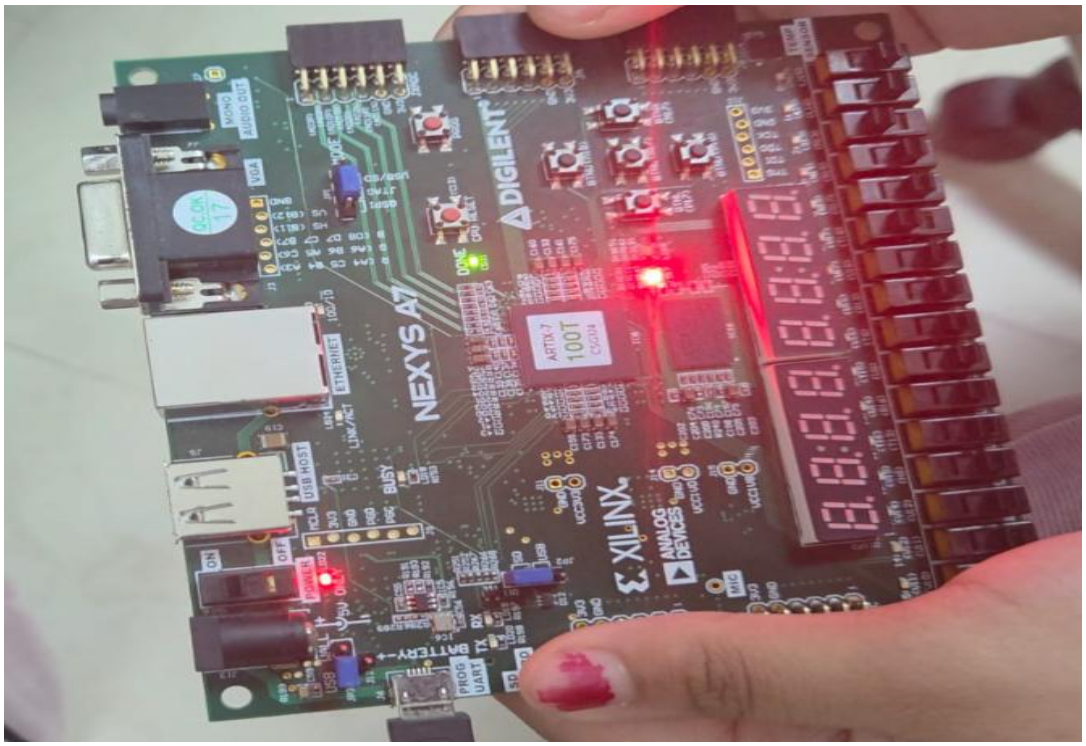
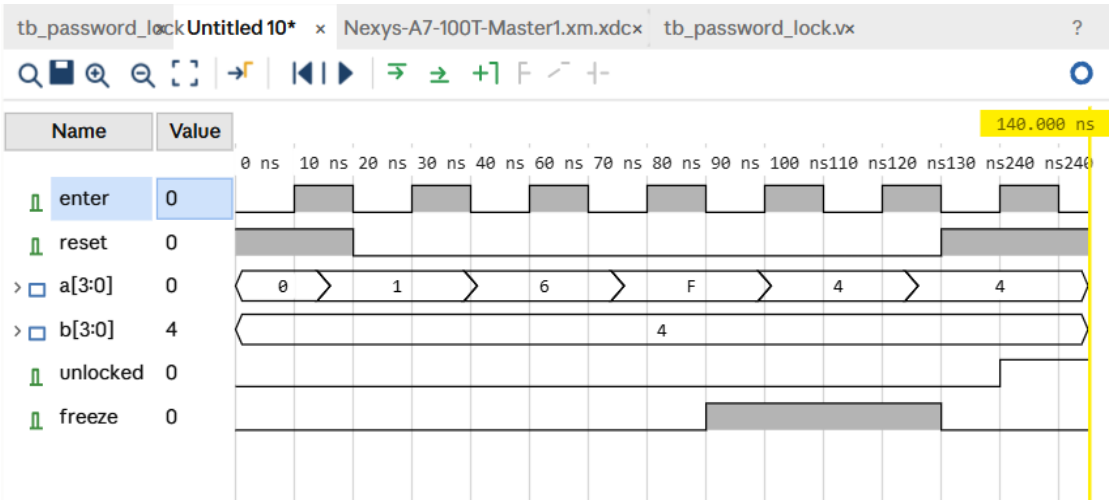



Fig. Output for incorrect password attempt for more than 3 times

## Simulation Result:

### a. Output waveforms




### b. Console Output

```
# run 1000ns

--- Applying System Reset ---
Time=0 | Reset=1 | Enter=0 | Target(b)=0100 | Input(a)=0000 | Unlocked=0 | Freeze=0
Time=20000 | Reset=0 | Enter=0 | Target(b)=0100 | Input(a)=0000 | Unlocked=0 | Freeze=0

--- Attempt 1: Incorrect Password ---
Time=30000 | Reset=0 | Enter=0 | Target(b)=0100 | Input(a)=0001 | Unlocked=0 | Freeze=0
Time=40000 | Reset=0 | Enter=1 | Target(b)=0100 | Input(a)=0001 | Unlocked=0 | Freeze=0
Time=50000 | Reset=0 | Enter=0 | Target(b)=0100 | Input(a)=0001 | Unlocked=0 | Freeze=0

--- Attempt 2: Incorrect Password ---
Time=60000 | Reset=0 | Enter=0 | Target(b)=0100 | Input(a)=0110 | Unlocked=0 | Freeze=0
Time=70000 | Reset=0 | Enter=1 | Target(b)=0100 | Input(a)=0110 | Unlocked=0 | Freeze=0
Time=80000 | Reset=0 | Enter=0 | Target(b)=0100 | Input(a)=0110 | Unlocked=0 | Freeze=0

--- Attempt 3: Incorrect Password (Should Freeze) ---
Time=90000 | Reset=0 | Enter=0 | Target(b)=0100 | Input(a)=1111 | Unlocked=0 | Freeze=0
Time=100000 | Reset=0 | Enter=1 | Target(b)=0100 | Input(a)=1111 | Unlocked=0 | Freeze=0
Time=110000 | Reset=0 | Enter=0 | Target(b)=0100 | Input(a)=1111 | Unlocked=0 | Freeze=0

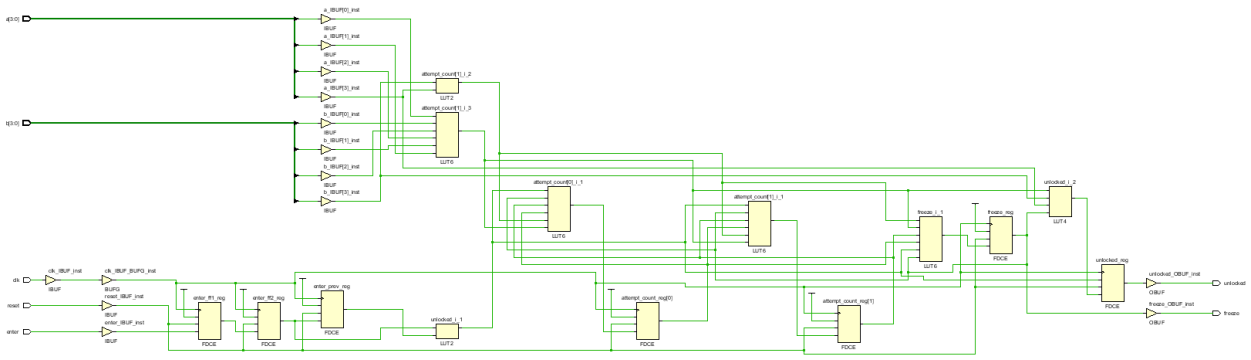
--- Entering Correct Password While System is Frozen ---
Time=140000 | Reset=0 | Enter=0 | Target(b)=0100 | Input(a)=0100 | Unlocked=0 | Freeze=0
Time=150000 | Reset=0 | Enter=1 | Target(b)=0100 | Input(a)=0100 | Unlocked=0 | Freeze=0
Time=160000 | Reset=0 | Enter=0 | Target(b)=0100 | Input(a)=0100 | Unlocked=0 | Freeze=0

--- Resetting System to Clear Freeze State ---
Time=190000 | Reset=1 | Enter=0 | Target(b)=0100 | Input(a)=0100 | Unlocked=0 | Freeze=0
Time=210000 | Reset=0 | Enter=0 | Target(b)=0100 | Input(a)=0100 | Unlocked=0 | Freeze=0

--- Attempting Correct Password After Reset ---
Time=230000 | Reset=0 | Enter=1 | Target(b)=0100 | Input(a)=0100 | Unlocked=0 | Freeze=0
Time=240000 | Reset=0 | Enter=0 | Target(b)=0100 | Input(a)=0100 | Unlocked=0 | Freeze=0

Simulation Complete
```

## c. RTL Schematic Diagram



## d. Synthesis Report

Tcl Console | Messages | Log | Reports | Design Runs | Power | DRC | Methodology | Timing | Utilization

| Report | Report Type | Options | Modified | Size |
|---|---|---|---|---|
| Synthesis | | | | |
| Synth Design (synth_design) | | | | |
| synth_1_synth_report_utilization_0 | Report on utilization of resources on the targeted device (report_utilization) | | 10/5/26, 2:12 PM | 6.6 KB |
| synth_1_synth_synthesis_report_0 | Vivado Synthesis Report | | 10/5/26, 2:12 PM | 18.3 KB |
| Implementation | | | | |
| impl_1 | | | | |
| Design Initialization (init_design) | | | | |
| impl_1_init_report_timing_summary_0 | Report timing summary (report_timing_summary) | | | |
| Opt Design (opt_design) | | | | |
| impl_1_opt_report_drc_0 | Report on error or violations against a set of design rule checks (report_drc) | | 10/5/26, 2:13 PM | 4.4 KB |
| impl_1_opt_report_timing_summary_0 | Report timing summary (report_timing_summary) | | | |
| Power Opt Design (power_opt_design) | | | | |
| impl_1_power_opt_report_timing_summary_0 | Report timing summary (report_timing_summary) | | | |
| Place Design (place_design) | | | | |
| impl_1_place_report_io_0 | Report information about all the IO sites on the device (report_io) | | 10/5/26, 2:13 PM | 96.7 KB |
| impl_1_place_report_utilization_0 | Report on utilization of resources on the targeted device (report_utilization) | | 10/5/26, 2:13 PM | 8.4 KB |

## Inference:

The experimental observations confirm that the password verification and security logic function correctly. The system distinguishes between valid and invalid password entries, keeps track of the number of attempts, and activates the lock after three unsuccessful attempts. The reset input successfully restores the system to its normal initial condition.

## Transistor Count Comparison per Gate/Sub-module

| Gate / Module | Static CMOS Transistor Count | Pass Transistor Logic / Transmission Gate Count | Reduction |
|---|---|---|---|
| **EXNOR Gate** | **12** (8-Tx core + 4-Tx inverters for $A, B$) | **6** (2-Tx TG + 4-Tx inverters for $A, B$) | **50%** |
| **EXOR Gate** | **12** (8-Tx core + 4-Tx inverters for $A, B$) | **6** (2-Tx TG + 4-Tx inverters for $A, B$) | **50%** |
| **2-Input AND Gate** | **6** (4-Tx NAND + 2-Tx Inverter) | **4** (2-Tx MUX/Pass logic + 2-Tx Inverter) | **33.3%** |
| **2-Input OR Gate** | **6** (4-Tx NOR + 2-Tx Inverter) | **4** (Pass Transistor implementation) | **33.3%** |
| **2-Input NAND Gate** | **4** | **4** (TG implementation) | **0%** |
| **NOT Gate (Inverter)** | **2** | **2** | **0%** |
| **D Flip-Flop** | **22 – 24** | **12 – 16** (TG Master-Slave DFF) | **~36%** |
| **T Flip-Flop** | **28 – 32** | **16 – 20** (TG Master-Slave TFF) | **~38%** |

**Total Circuit Transistor Breakdown (Password Lock System)**

Based on the top-level schematic containing:

- **4 × EXNOR Gates** (4-bit password comparison:
- **6 × 2-input AND Gates**
- **1 × 2-input OR Gate**
- **1 × NOT Gate**
- **2 × D Flip-Flops**
- **2 × T Flip-Flops**

| Implementation Style | Sub-Component Breakdown | Total Transistor Count |
|---|---|---|
| Static CMOS Implementation | $(4 \times 12) + (6 \times 6) + 6 + 2 + (2 \times 24) + (2 \times 32)$ | 204 Transistors |
| Pass Transistor / TG Implementation | $(4 \times 6) + (6 \times 4) + 4 + 2 + (2 \times 16) + (2 \times 20)$ | 126 Transistors |

**Net Savings for the Password Lock System: 78 Transistors (~38.2% total area reduction)** using Transmission Gate / Pass Transistor Logic.

## Result:

The 4-bit digital password lock system was successfully designed and implemented using Verilog HDL in Vivado on the FPGA board. The hardware outputs were verified using the LEDs, where the Unlocked LED indicates a correct password and the Locked LED indicates that the maximum number of incorrect attempts has been reached. Thus, the required password-based security operation was successfully achieved.